SOVEREIGN SYSTEM SPECIFICATION

Infinite Memory Architecture

A bounded 8K attention window backed by durable, selective, and self-maintaining memory for a seamless local companion experience.

TARGET	Google Pixel 10 Pro / Termux / LiteRT-LM
MODEL	Gemma 4 E4B-it / GPU
STATUS	Design baseline - ready for implementation
DATE	24 August 2026

Prepared for Solkara // Project Intermix

01 / DESIGN INTENT

Design intent

Project Intermix should feel as if its memory is effectively infinite while keeping the physical model context bounded, predictable, and safe for mobile inference. The system achieves continuity through durable storage, selective recall, session checkpoints, and background consolidation - not by replaying the entire transcript.

CORE PRINCIPLE	The 8K model window is attention. SQLite is memory. Retrieval decides what the Sovereign Core remembers right now.

Target experience

- Close and reopen the TUI without losing conversational continuity.
- Recall hobbies, preferences, personal context, code decisions, and unfinished threads when relevant.
- Keep normal conversation, live web evidence, and code-agent execution as separate context domains.
- Update memory quietly during idle moments without competing with active E4B generation.
- Keep complete history viewable in the cockpit without injecting complete history into the prompt.

Non-goals

- The database is not a verbatim prompt replay system.
- Every sentence is not promoted into long-term memory.
- Unverified inferences are not treated as permanent personal facts.
- A cold 32K context is not required to produce long-lived continuity.

Constraint	Design response
Mobile RAM and KV-cache growth	Bound the runtime to 8K and reserve output capacity.
E4B consolidation cost	Queue short consolidation passes and run them only while the engine is idle.
Thermal load	Peltier cooling supports sustained inference; prompt budgeting still controls memory pressure.
Environment hopping	Persist architecture, schema version, migrations, and human-readable exports.

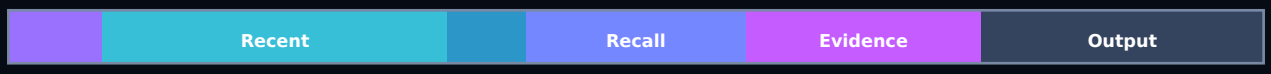
02 / LAYERED MEMORY MODEL

Layered memory model

The companion presents one continuous identity, but its memory is divided into layers with different lifetimes, retrieval rules, and trust levels.

Layer	Contents	Prompt policy
Identity	Persona, relationship style, core directives	Pinned; always present and strictly size-limited
Working	Current user request, recent turns, immediate task	Always present; evicted by token budget
Session	Narrative summary, active topics, open loops	Loaded at startup and refreshed at checkpoints
Personal	Preferences, hobbies, goals, explicit sensitive context	Retrieved by relevance, salience, and consent
Knowledge	Sourced world facts with freshness metadata	Only active while verified and unexpired
Project	Architecture decisions, file events, tests, unresolved work	Loaded for project discussion or /workspace

Runtime context assembly



Identity: 600
Recent: 2,200
Session: 500
Recall: 1,400
Evidence: 1,500
Output: 1,800

The budget is dynamic. Ordinary conversation may reassign unused evidence capacity to recent turns or output. Web and workspace modes must evict low-priority conversational material rather than exceed the cap. Token counts should come from the model tokenizer when available; conservative estimates are acceptable during the first implementation.

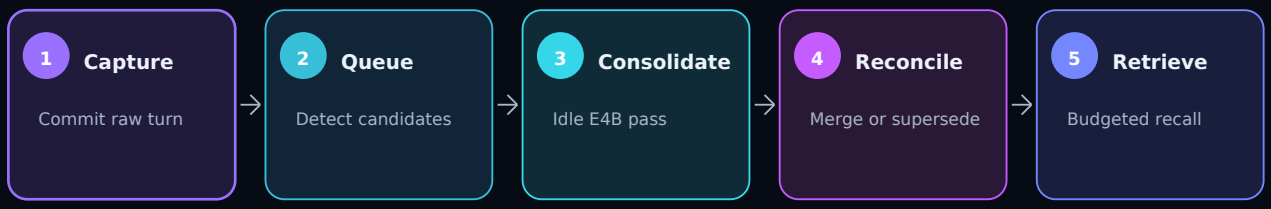
Context priority

Priority	Material	Eviction behavior
P0	Safety and core identity	Never evicted
P1	Current request and required tool evidence	Never evicted for the active turn
P2	Recent turns and active session state	Trim oldest first
P3	Retrieved memories	Remove low-scoring or redundant items
P4	Optional lore and atmospheric detail	Remove first under pressure

03 / MEMORY LIFECYCLE

Memory lifecycle

Raw history must be captured synchronously. Interpretation and consolidation happen later. This split makes shutdown safe and keeps the visible response path responsive.



Operating rules

- Commit every user and assistant turn before scheduling any derived memory work.
- Use inexpensive rules to identify obvious candidates such as preferences, explicit remember requests, named projects, and corrections.
- Run a short structured E4B consolidation pass every 3-5 turns, on topic changes, during idle time, or on clean shutdown.
- Never run chat generation and consolidation concurrently on the same mobile inference engine.
- If consolidation fails, keep the raw messages and retry later; conversation must not be lost.

Structured memory proposal

```
{
  "operation": "upsert",
  "kind": "user_preference",
  "key": "preferred_coding_style",
  "value": "modular lightweight Python",
  "confidence": 0.94,
  "importance": 0.72,
  "sensitive": false,
  "source_message_id": 418
}
```

Permitted operations

Operation	Meaning
create	Create a new durable memory with provenance.
reinforce	Increase confidence when independent conversation evidence agrees.
update	Revise a mutable detail while retaining its earlier version.
supersede	Deactivate a conflicting older memory and link the replacement.
expire	Mark time-sensitive knowledge unavailable until refreshed.
forget	Remove or tombstone a user-selected memory.
ignore	Reject an ephemeral, unsafe, redundant, or low-value candidate.

04 / STORAGE AND RETRIEVAL CONTRACT

Storage and retrieval contract

SQLite remains the canonical store. TXT and Markdown are transparent exports, migration inputs, and emergency recovery formats - never the live source of truth after migration.

Table	Minimum responsibility
schema_meta	Schema version, migration status, installation identity
sessions	Title, summary, state, active project, created and updated times
messages	Immutable role/content archive linked to a session
memories	Typed fact, confidence, salience, sensitivity, provenance, freshness
memory_revisions	Conflict history and supersession relationships
summaries	Source message range, summary type, token estimate
web_facts	Claim, source URL, retrieval time, expiry, verification state
project_events	Tool action, path, hashes, test result, short explanation
memory_jobs	Queued consolidation, retries, error state
FTS5 indexes	Searchable message, memory, summary, and project text

Core memory fields

`kind, normalized_key, value, source_message_id, confidence, salience, explicitly_stated, sensitive, created_at, updated_at, last_confirmed_at, last_used_at, expires_at, active, supersedes_id`

Retrieval score

RANKING	Combine query relevance, session continuity, salience, recency, and confirmation strength. Apply hard penalties to expired knowledge, sensitive records outside their allowed scope, and memories already represented by the session summary.

Signal	Purpose	Typical influence
Relevance	Does the memory address the current topic?	Highest
Continuity	Does it belong to the active session or project?	High
Salience	Would forgetting materially hurt the relationship or task?	Medium-high
Recency	Is a recent memory more likely to be current?	Medium
Confirmation	Was it explicit or reinforced across turns?	Medium
Freshness	Is a world fact still within its validity period?	Gate

05 / PERSONAL MEMORY AND HARD FACTS

Personal memory and hard facts

Personal context and external knowledge have different truth rules. They must never share an undifferentiated fact bucket.

Property	Personal memory	Hard/world knowledge
Authority	User statement is primary	Named external source is primary
Typical lifetime	Long-lived but revisable	Bound by freshness class and expiry
Conflict handling	Prefer newer explicit correction	Reverify against live sources
Sensitive data	Consent and visibility controls required	Usually not sensitive; source licensing still matters
Prompt form	Short relationship-aware fact	Claim plus source date and verification state

Sensitive-context policy

- Explicit user statements may become durable memory when relevant to future support.
- Model-inferred mental-health states remain temporary unless the user confirms them.
- The system does not invent or persist diagnoses.
- Every sensitive memory remains inspectable and individually removable.
- Retrieval excludes sensitive context when it is irrelevant to the current conversation.

Freshness-aware web behavior

- Forced /web always retrieves current evidence.
- Automatic grounding triggers for volatile intent: latest versions, prices, schedules, news, laws, current roles, and other time-sensitive claims.
- Search one provider first and use fallbacks only on failure or insufficient evidence.
- Cache evidence with a category-specific expiry instead of repeating identical searches.
- Expired knowledge becomes a retrieval trigger, not an injected fact.

TRUST RULE	A remembered claim without provenance may guide retrieval, but it must not masquerade as verified current knowledge.

06 / SESSION CONTINUITY AND COCKPIT

Session continuity and cockpit

The cockpit may display complete history while the model sees only a carefully assembled working set. Visible history and active context are intentionally different.

Clean shutdown checkpoint

Checkpoint field	Purpose
title	Human-readable session label for browsing
narrative_summary	Compressed conversational continuity
current_task	What the user and system were actively doing
open_loops	Unresolved questions, promised follow-ups, failing tests
decisions	Architecture or preference choices that should persist
tone_state	Minimal conversational atmosphere; never a diagnosis
last_message_ids	Pointers for loading the last 2-4 raw turns
active_project	Project-scoped retrieval anchor

Startup rehydration

Load identity, the latest checkpoint, the last 2-4 turns, and a small relevant-memory set. Render older messages from storage when the user scrolls; do not add them to the prompt merely because they are visible.

Command surface

Command	Behavior
/status	Engine, context utilization, queue state, and active session
/inspect context	Show exactly which memory and evidence blocks entered the prompt
/memory facts	Browse durable facts with source and confidence
/memory search topic	Search archived and consolidated memory
/memory pin	Protect an important memory from automatic expiry
/memory forget id	Remove a selected memory
/sessions	Browse, name, resume, or export sessions
/web	Force live grounding
/workspace	Enter isolated code-agent mode
/clear	Clear the rendered conversation without deleting memory
/export	Produce portable Markdown or TXT history and memory reports

07 / SUBSYSTEM BOUNDARIES

Subsystem boundaries

Chat, web grounding, memory maintenance, and code execution share an identity but not an execution path. Keeping these domains separate prevents code logs and search noise from consuming ordinary conversational attention.

Subsystem	Owns	May write to memory
Conversation	Visible response, atmosphere, dialogue continuity	Raw turns and proposed personal/session memories
Memory worker	Consolidation, conflicts, summaries, expiry	Validated memories, revisions, checkpoints
Web router	Freshness detection, provider fallback, cache	Sourced and expiring web facts
Workspace agent	File tools, execution, repair loop	Project events and final task summary
TUI	Rendering, commands, history browser, inspection	User-initiated pin, forget, rename, export

Workspace isolation

- Normal chat receives only concise project summaries unless code context is explicitly relevant.
- /workspace owns its own prompt budget, tool protocol, timeout, and bounded repair loop.
- Every successful or failed tool action creates an objective event: action, path, hashes, command, exit code, and output summary.
- Self-documentation is generated from events, not reconstructed from the model's conversational claims.
- The bordered Agent Sandbox Shell remains visually distinct, matching the existing cockpit hierarchy.

Proposed module boundary

```
engine/
  llm_controller.py     # orchestration only
  memory_store.py       # schema and transactions
  memory_retriever.py   # FTS5 ranking and selection
  memory_worker.py      # queued consolidation
  prompt_builder.py      # 8K budgeting
  session_manager.py    # checkpoint and rehydrate
  web_search.py        # grounded retrieval
  agent.py             # isolated workspace tools
  tui_app.py           # cockpit surface
```

08 / IMPLEMENTATION SEQUENCE

Implementation sequence

Implementation should proceed in small reversible milestones. The existing convo.txt and sovereign.db remain untouched until the new schema passes migration and rehydration tests.

Phase	Deliverable	Exit condition
0 - Preserve	Timestamped backups and read-only inventory	Existing chats recoverable
1 - Store	Versioned SQLite schema and immutable message writes	Restart loses no completed turn
2 - Rehydrate	Sessions, checkpoints, history browser	Closing and reopening restores continuity
3 - Retrieve	FTS5 index, ranking, 8K prompt builder	Database growth no longer grows prompt size
4 - Consolidate	Idle queue and structured memory operations	Facts update without blocking chat
5 - Ground	Freshness classes and web cache	Expired facts trigger verification
6 - Integrate	Project events and workspace summaries	Code logs remain outside normal chat
7 - Surface	Memory commands, context inspector, exports	Every recall is debuggable and removable

Acceptance tests

- After 500 archived messages, the assembled prompt remains within the configured 8K budget.
- A hobby mentioned many sessions ago is recalled when the topic returns, but not injected into unrelated requests.
- A corrected preference supersedes the old value without erasing provenance.
- A stale software-version fact triggers web verification rather than being presented as current.
- A forced process stop can lose at most an in-flight generation, never an already committed turn.
- Closing and reopening restores the previous task, open loops, and last conversational turns.
- Workspace errors and repair attempts appear in the sandbox panel and project history, not the ordinary prompt.
- Sensitive memories are inspectable, scoped, and removable.

NEXT ACTION	Inspect the current TUI and web-search interfaces, then implement Phase 0 and Phase 1 without altering the active launcher until validation succeeds.

Portable handoff checklist

- This specification accompanies the source tree when changing assistants or environments.
- Record the installed litert-lm version and command-line capabilities before modifying inference orchestration.
- Keep schema migrations deterministic, numbered, and safe to rerun.
- Export a Markdown memory report before every structural migration.

Reference anchors

Android memory management: developer.android.com/topic/performance/memory
SQLite FTS5: sqlite.org/fts5.html
LiteRT-LM: github.com/google-ai-edge/LiteRT-LM
Gemma model documentation: ai.google.dev/gemma/docs/core